Módulo 2 – SQL

Resumo de estudo: da visão geral aos JOINs

Conteúdo:

• Visão geral do SQL

• DDL vs. DML

• SELECT

• INSERT, UPDATE, DELETE

• WHERE

• ORDER BY

• GROUP BY e agregação

• INNER JOIN, LEFT JOIN, RIGHT JOIN

Os exemplos usam duas tabelas: **clientes**(id, nome, cidade) e **pedidos**(id, cliente_id, valor, data).

1. Visão geral do SQL

SQL (*Structured Query Language*) é a linguagem padrão para definir, manipular e consultar dados em bases de dados relacionais (PostgreSQL, MySQL, SQL Server, Oracle, SQLite). Os dados organizam-se em **tabelas** (linhas = registos, colunas = atributos), relacionadas por **chaves primárias** e **chaves estrangeiras**.

SQL é declarativo: descreve-se *o que* se quer, e o motor decide *como* obtê-lo.

2. DDL vs. DML

	DDL – Data Definition Language	DML – Data Manipulation Language
Objetivo	Definir/alterar a estrutura	Manipular os dados
Comandos	CREATE, ALTER, DROP, TRUNCATE	SELECT, INSERT, UPDATE, DELETE
Afeta	Tabelas, índices, vistas, esquemas	Linhas dentro das tabelas

```
-- DDL
CREATE TABLE clientes (
  id     INT PRIMARY KEY,
  nome   VARCHAR(100) NOT NULL,
  cidade VARCHAR(50)
);
ALTER TABLE clientes ADD email VARCHAR(100);
DROP TABLE clientes;

-- DML
INSERT INTO clientes (id, nome, cidade) VALUES (1, 'Ana', 'Lisboa');
```

3. SELECT

Obtém dados de uma ou mais tabelas. Sintaxe geral e ordem de escrita:

```
SELECT   colunas
FROM     tabela
WHERE    condição
GROUP BY colunas
HAVING   condição_sobre_grupos
ORDER BY colunas;

SELECT * FROM clientes;
SELECT nome, cidade FROM clientes;
SELECT DISTINCT cidade FROM clientes;
SELECT nome AS cliente FROM clientes;   -- alias
SELECT * FROM clientes LIMIT 10;        -- (TOP 10 no SQL Server)
```

4. INSERT, UPDATE, DELETE

INSERT – inserir registos

```
INSERT INTO clientes (id, nome, cidade)
VALUES (2, 'Rui', 'Porto'), (3, 'Eva', 'Faro');
```

UPDATE – alterar registos

```
UPDATE clientes SET cidade = 'Braga' WHERE id = 2;
```

DELETE – remover registos

```
DELETE FROM clientes WHERE id = 3;
```

Atenção: sem `WHERE`, UPDATE e DELETE afetam **todas** as linhas. Teste primeiro com um SELECT usando a mesma condição.

5. WHERE

Filtra linhas antes de qualquer agrupamento.

Operador	Exemplo
=, <>, <, >, <=, >=	valor > 100
AND, OR, NOT	cidade = 'Lisboa' AND valor > 50
BETWEEN	valor BETWEEN 10 AND 50
IN	cidade IN ('Lisboa','Porto')
LIKE (% e _)	nome LIKE 'A%'
IS NULL / IS NOT NULL	email IS NULL

```
SELECT * FROM pedidos WHERE valor BETWEEN 50 AND 200 AND data >= '2025-01-01';
```

6. ORDER BY

Ordena o resultado: `ASC` (por omissão) ou `DESC`. Admite várias colunas.

```
SELECT * FROM pedidos ORDER BY valor DESC;
SELECT * FROM clientes ORDER BY cidade ASC, nome ASC;
```

7. GROUP BY e agregação

Funções de agregação reduzem várias linhas a um valor:

Função	Descrição
COUNT()	Número de linhas (COUNT(*)) ou valores não nulos
SUM()	Soma
AVG()	Média
MIN() / MAX()	Mínimo / máximo

`GROUP BY` divide as linhas em grupos e aplica a agregação a cada um. `HAVING` filtra os grupos (WHERE filtra linhas, HAVING filtra grupos).

```
SELECT cliente_id,
       COUNT(*)   AS n_pedidos,
       SUM(valor) AS total,
       AVG(valor) AS media
FROM pedidos
GROUP BY cliente_id
HAVING SUM(valor) > 500
ORDER BY total DESC;
```

Regra: toda a coluna no SELECT que não seja agregada tem de constar no GROUP BY.

8. JOINs

Combinam linhas de tabelas diferentes com base numa condição, normalmente chave estrangeira = chave primária.

Tipo	Resultado
INNER JOIN	Apenas linhas com correspondência nas duas tabelas
LEFT JOIN	Todas as linhas da tabela da esquerda; NULL onde não há correspondência à direita
RIGHT JOIN	Todas as linhas da tabela da direita; NULL onde não há correspondência à esquerda

INNER JOIN

```
SELECT c.nome, p.valor
FROM clientes c
INNER JOIN pedidos p ON p.cliente_id = c.id;
```

LEFT JOIN

Útil para encontrar registos sem relação, por exemplo clientes sem pedidos:

```
SELECT c.nome, p.id AS pedido
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

RIGHT JOIN

```
SELECT c.nome, p.valor
FROM clientes c
RIGHT JOIN pedidos p ON p.cliente_id = c.id;
```

RIGHT JOIN equivale a um LEFT JOIN com as tabelas trocadas; na prática usa-se mais o LEFT JOIN.

JOIN com agregação

```
SELECT c.nome, COUNT(p.id) AS n_pedidos, COALESCE(SUM(p.valor),0) AS total
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.nome;
```

Ordem de execução lógica

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Explica porque não se pode usar um alias do SELECT no WHERE.